SCALING DISTRIBUTED MACHINE LEARNING WITH THE PARAMETER SERVER

Agenda

Background - Parallelism Concepts

Content – Description of Their Model

Evaluation – Case Studies

Main Ideas:

Range Based Communication

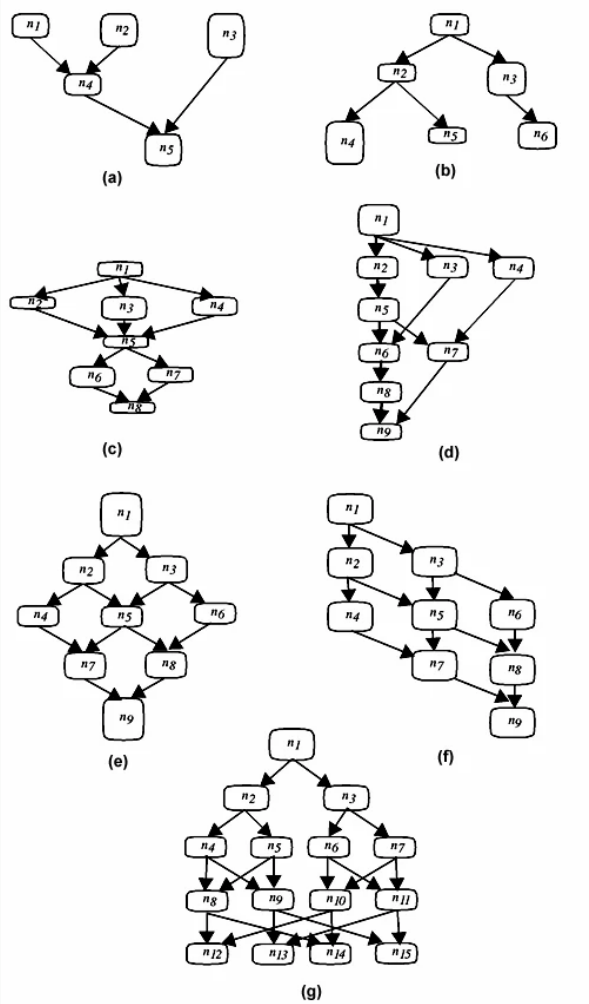
"Flexible Consistency"

Task Graphs

Chapter 3 of our textbook goes over different thread management schemes (worker pools, pipelining, task graphs, ...etc.).

As covered in a previous lecture, Task Graphs have dependency graphs or DAGs.

Dependency DAG is unrolled by the scheduler.

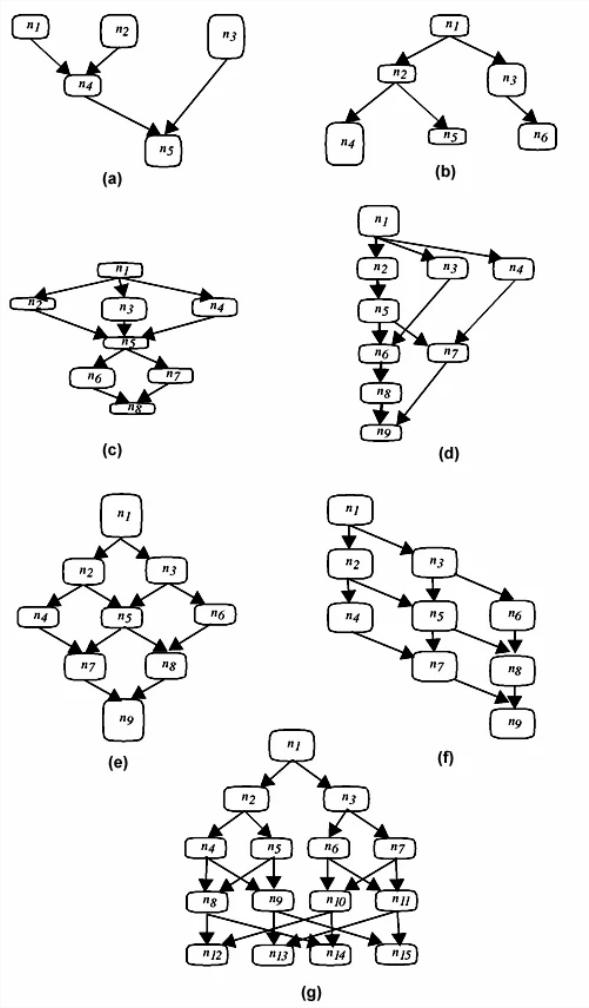


Task Graphs

Task Graphs are a general solution.

Popular for cross platform, multiple use, and academic purposes because they only need to be coded once.

Note that performance is inconsistent per context.

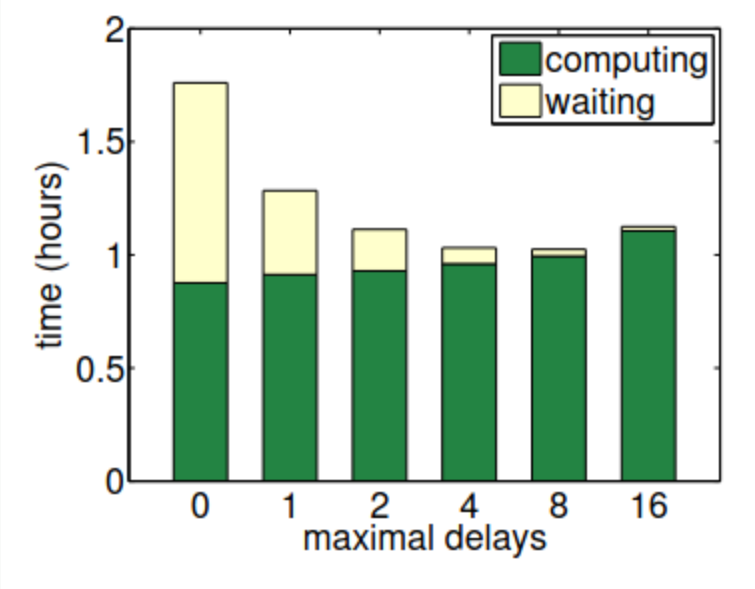


Task Graphs

Performance inconsistency can arise from hardware issues, dependency issues, or varying workloads.

It is common for Task graphs to have a workload balancing mechanism as an optimization.

In this paper, this key concept relates to their term ‘flexible consistency,’ but is not used for optimization.

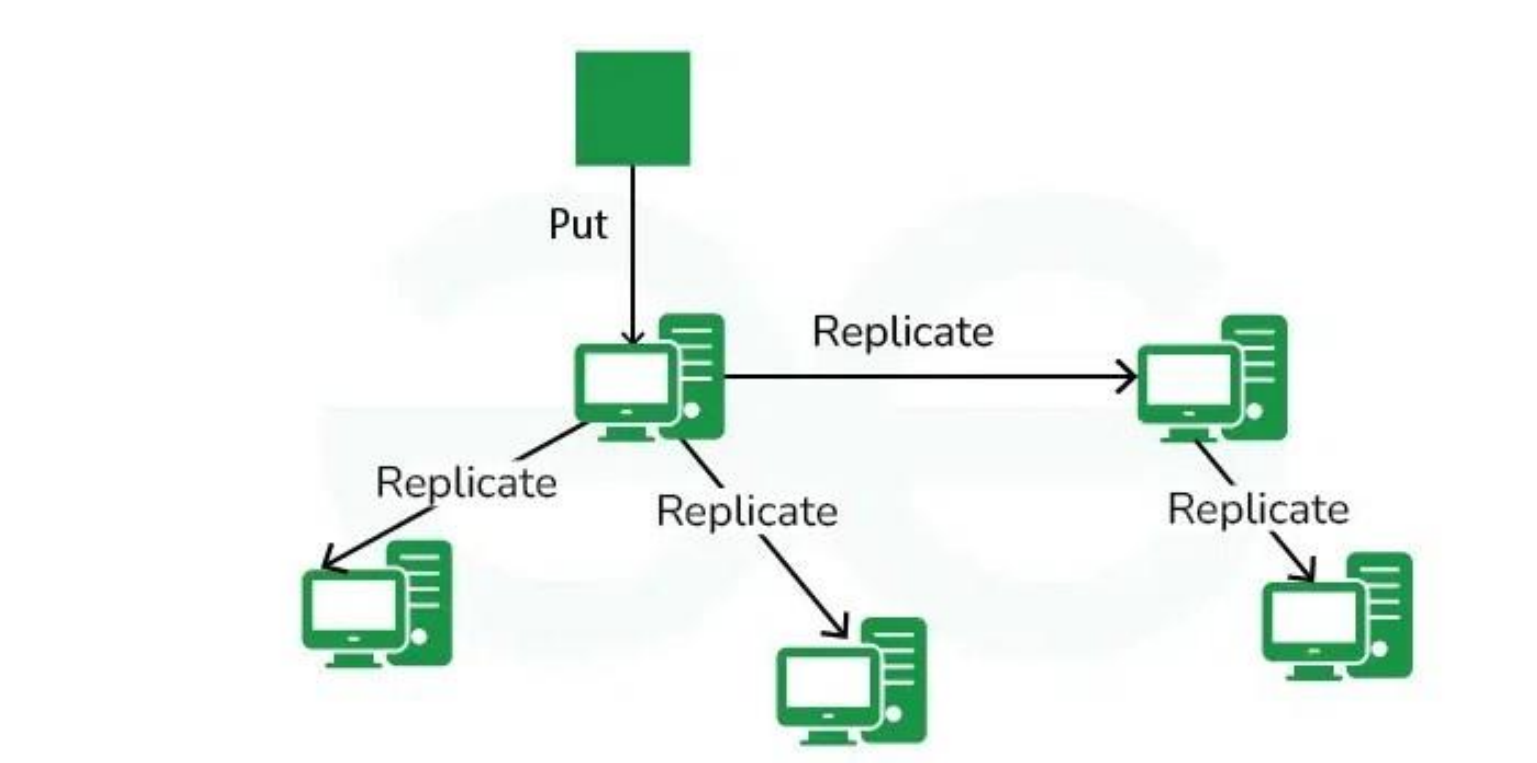


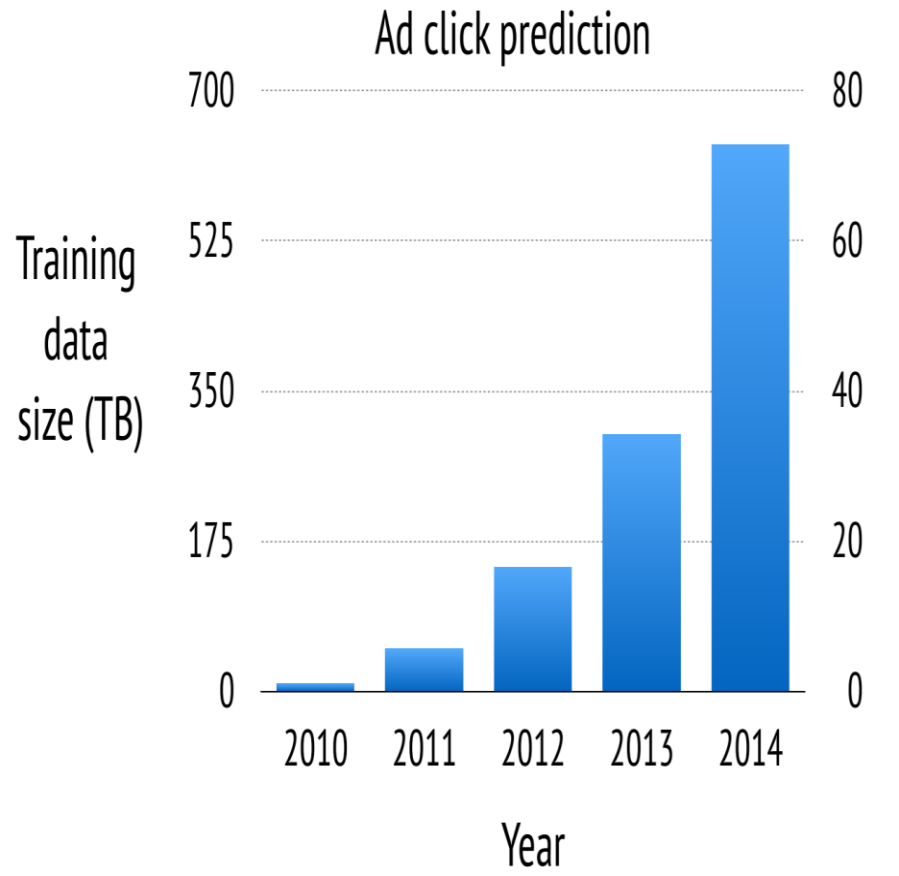
Consistency in Distributed Systems

The consistency model in a distributed system defines criteria for data synchronization.

Defines how data is accessed / changed / updated across many nodes.

This also relates to their key concept of ‘flexible consistency.’





Motivations

Already in 2014, it was common to see ML models require on the order of 10^9 and 10^{12} number of parameters. Thus, Parameter Servers manage incredibly large amounts of globally shared data amongst workers.

The 3 major challenges they address in their paper:

- These systems require enormous bandwidth
- The cost of synchronization is high
- Fault tolerance is critical

Architecture

- Server Group
  - *Contains the Server Nodes and Manager Node*
- Server Manager Node
  - *Assigns parameter partitions to Server Nodes*
- Server Node
  - *Maintains a parameter partition*
  - *Communicates with other Server Nodes for purposes of Fault Tolerance (spoiler alert)*

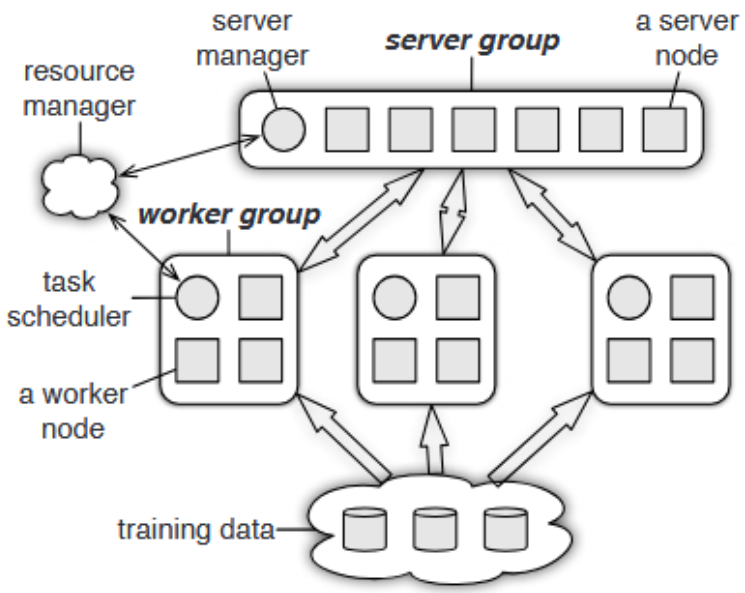


Figure 4: Architecture of a parameter server communicating with several groups of workers.

Architecture

- Worker Group
  - *Runs an application*
- Worker
  - *Stores local portion of training data for computation*
- Workers
  - *Communicate only with Server Nodes, never each other*
- Scheduler Node
  - *Assigns tasks and optimizes progress*

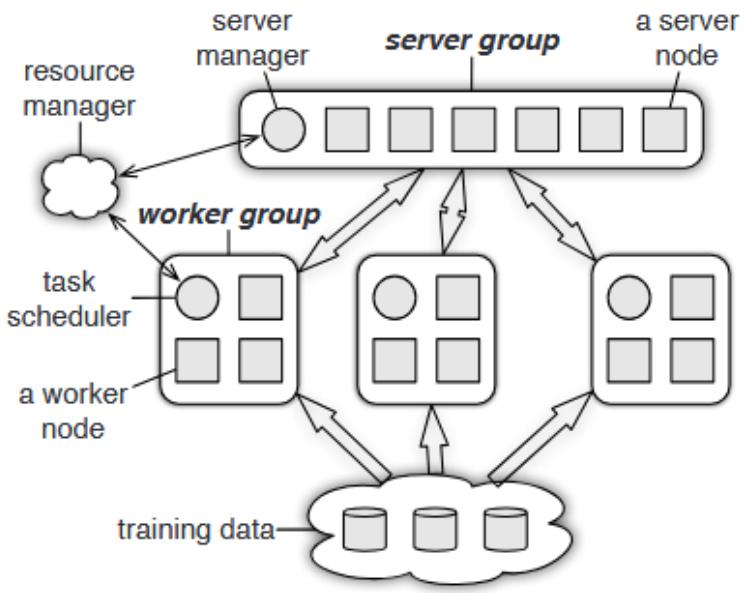
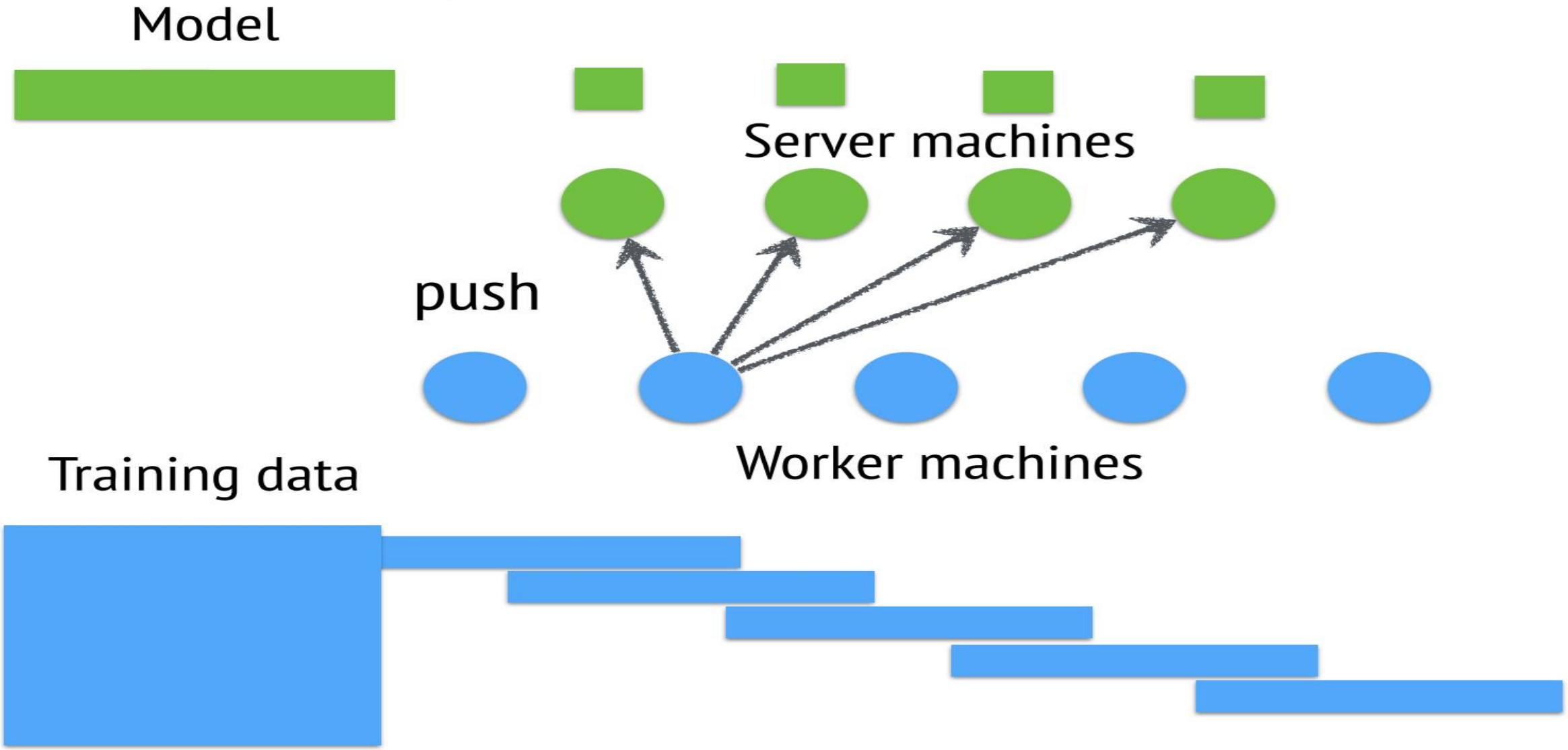


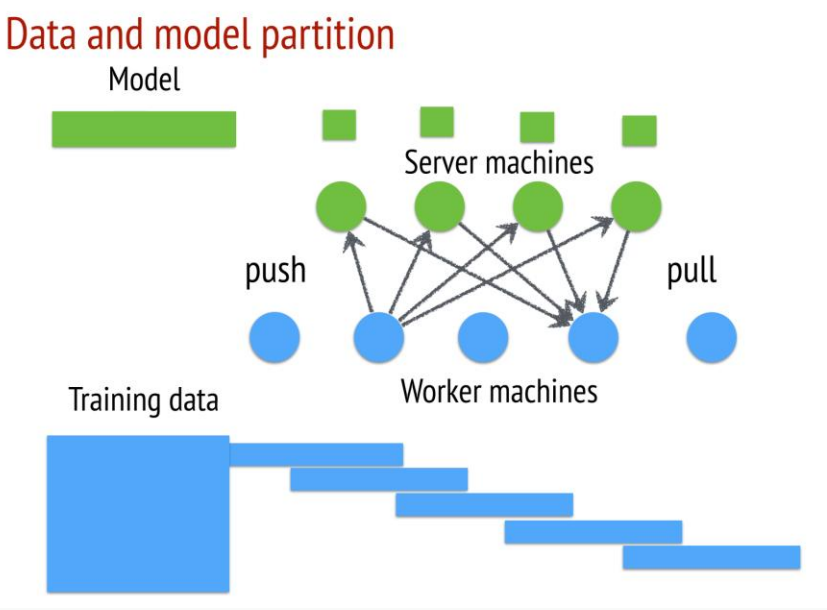
Figure 4: Architecture of a parameter server communicating with several groups of workers.

Data and model partition



Ranged Base Communication

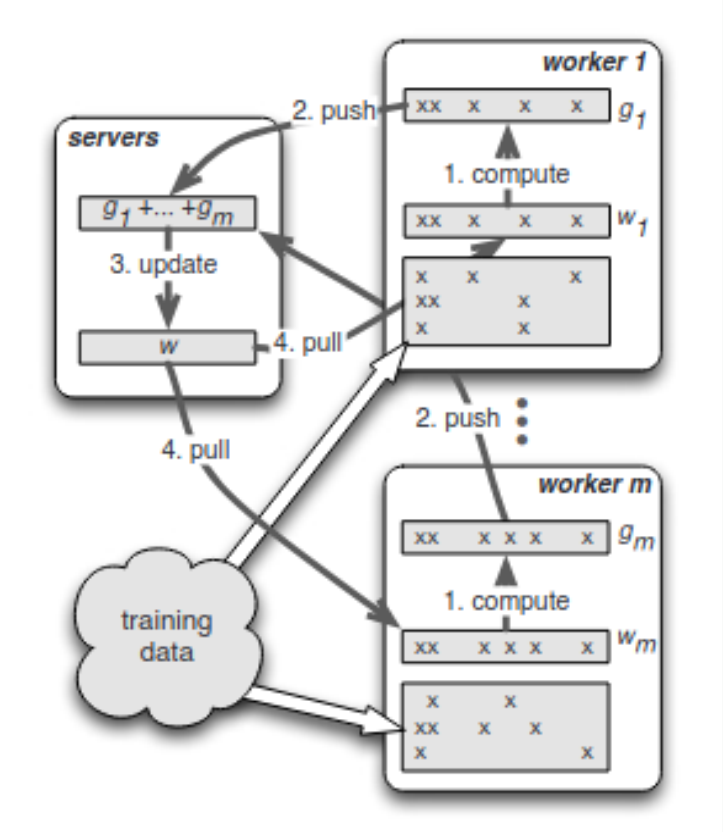
- Their Data Structure of parameters is similar to that of a hashmap of the form (Key, Value), ordered by key.
- ML models use Vectorized data. To take advantage of this fact, the data structures are also Vectorized.
- Pull/Push commands (seen on right) load/update parameters between workers and servers.
- This batches data to take advantage of network bandwidth, therefore parallelizing communications.
- According to the authors, this is their most impactful optimization.



Example Implementation

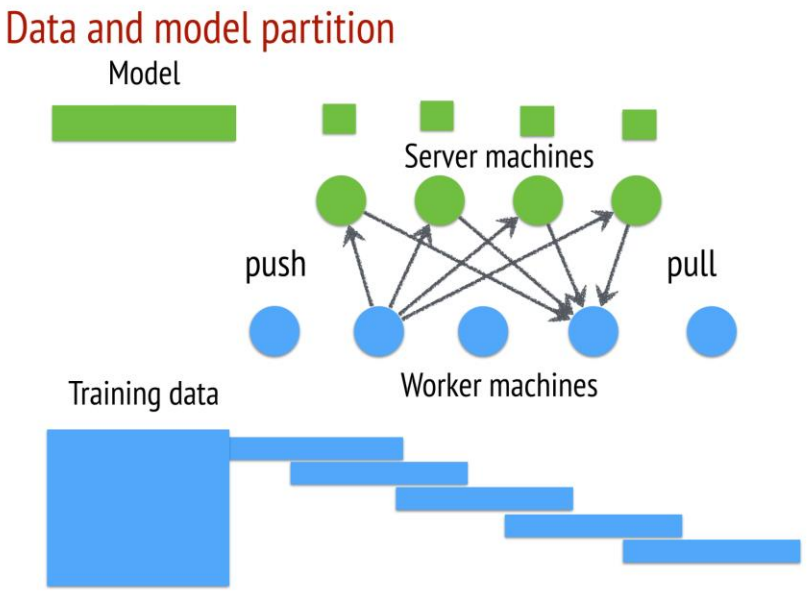
Algorithm 1:

- Task Scheduler: tell workers to load data and iterate
- Each worker caches w_r, the working set of parameter vectors, rather than all parameters
- Workers: calculate gradients, g_r; push gradients g_r; and pull updated parameter vectors, w_r
- Servers: aggregate gradients, g_r



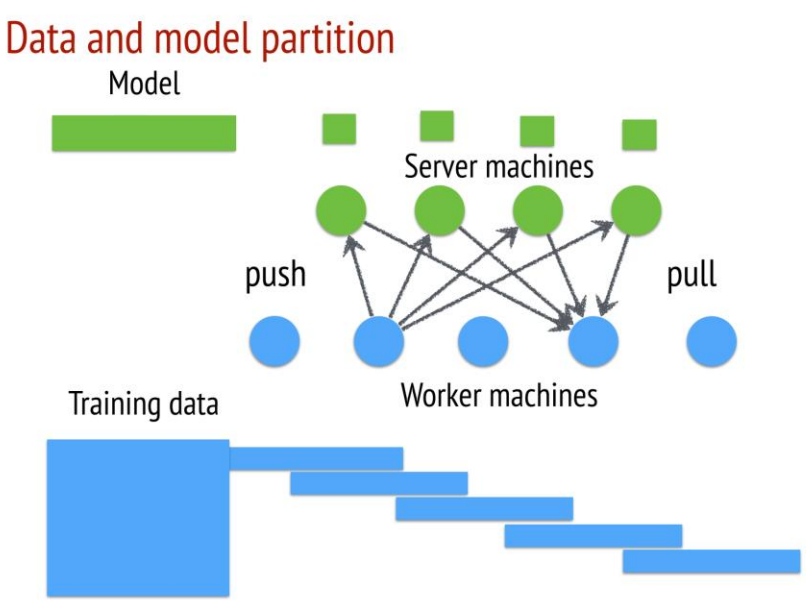
Data Inconsistency

- With regards to the image on the right, note that the training data exists on multiple nodes, i.e. the data can be on more than one node at a time.
- What happens when data is updated in different ways on multiple nodes and then gets pushed to the system? Which version of that data is correct?
- This is an example of ‘data inconsistency’ for a distributed system.



Data Inconsistency

- Data inconsistency can arise in many different ways depending on context.
- This paper addresses this issue using a dependency DAG in the scheduler.
- Therefore, this is a Task Graph as we defined earlier.



Flexible Consistency

- This is more complicated in the paper. Basically they have a threshold of scheduled delay, τ.
- Sequential – $\tau = 0$, everything is dependent, tasks are scheduled one at a time with no parallelization
- Eventual – $\tau = \infty$, nothing is dependent, schedule all tasks immediately, order does not matter
- Bounded Delay – new tasks are blocked until τ time has passed

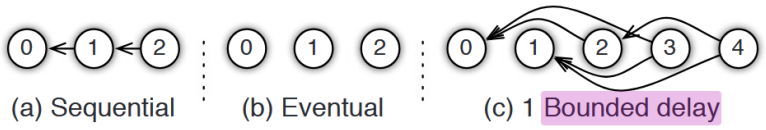


Figure 6: Directed acyclic graphs for different consistency models. The size of the DAG increases with the delay.

Assistive Structures for Implementations

Vector Clock – Used to make time stamps for the purpose of optimizing τ. Although, this can create tons of synchronization overhead!

Messages/Consistent Hashing/Replication – Used to select ‘key’s from the parameter data structures when calling push/pull. Which scheme to pick depends on the algorithm.

Server Management – The Server Manager Node and the Server nodes communicate with one another to handle situations when nodes drop or get added to the system.

Vector Clock Implementation

Algorithm 2 Set vector clock to t for range $\mathcal{R}$ and node i

```
for S ∈ {S_i : S_i ∩ R ≠ ∅, i = 1, ..., n} do
    if S ⊆ R then vc_i(S) ← t else
        a ← max(S^b, R^b) and b ← min(S^e, R^e)
        split range S into [S^b, a), [a, b), [b, S^e)
        vc_i([a, b)) ← t
    end if
end for
```

Sparse Logistic Regression

L1 regularization to promote sparsity

BUT non-smoothness is difficult

"[nobody] can scale [this] to the scale described in this paper"

	Method	Consistency	LOC
System A	L-BFGS	Sequential	10,000
System B	Block PG	Sequential	30,000
Parameter Server	Block PG	Bounded Delay KKT Filter	300

Table 3: Systems evaluated.

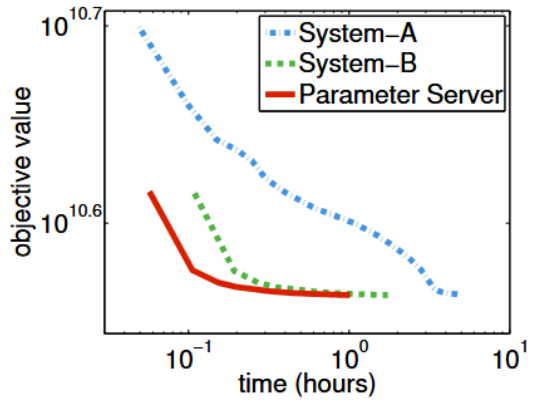


Figure 9: Convergence of sparse logistic regression. The goal is to minimize the objective rapidly.

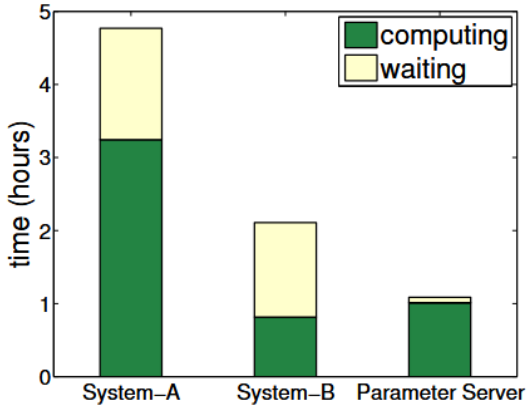


Figure 10: Time per worker spent on computation and waiting during sparse logistic regression.

L1 Regularization "Lasso Regression"

- Absolute values of coefficients as Ω
- Promotes sparsity, feature selection
- Contrasted to L2 regularization which promotes small values by adding the square of each coefficient

$$F(w) = \sum_{i=1}^{n} \ell(x_i, y_i, w) + \Omega(w).$$

Latent Dirichlet Allocation

Requires training, though not a DNN

Text topic classifications

Topic name	# Top urls
Programming	stackoverflow.com w3schools.com cplusplus.com github.com tutorialspoint.com jquery.com codeproject.com oracle.com qt-project.org bytes.com android.com mysql.com
Music	ultimate-guitar.com guitaretab.com 911tabs.com e-chords.com songsterr.com chordify.net musicnotes.com ukulele-tabs.com
Baby Related	babycenter.com whattoexpect.com babycentre.co.uk circleofmoms.com thebump.com parents.com momtastic.com parenting.com americanpregnancy.org kidshealth.org
Strength Training	bodybuilding.com muscleandfitness.com mensfitness.com menshealth.com t-nation.com livestrong.com muscleandstrength.com myfitnesspal.com elitefitness.com crossfit.com steroid.com gnc.com askmen.com

Table 4: Example topics learned using LDA over the .5 billion dataset. Each topic represents a user interest

Latent Dirichlet Allocation

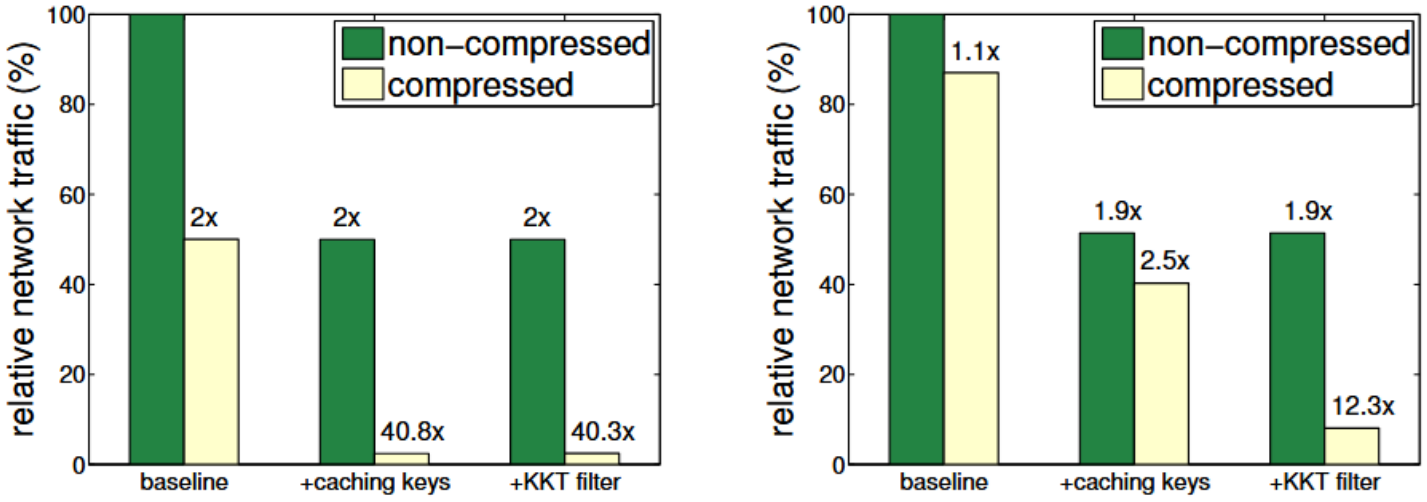


Figure 11: The savings of outgoing network traffic by different components. Left: per server. Right: per worker.

Latent Dirichlet Allocation

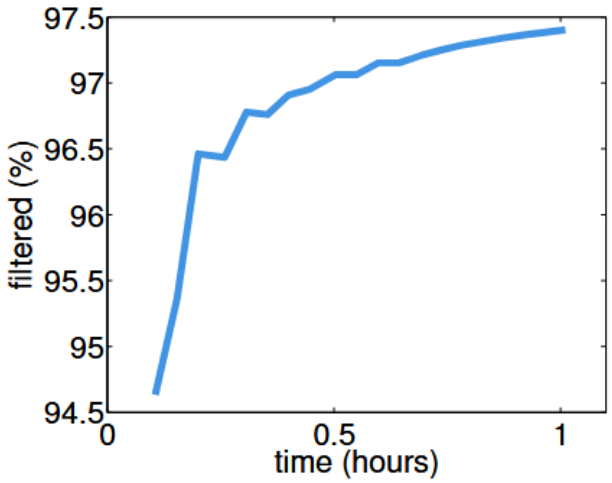


Figure 12: Unique features (keys) filtered by the KKT filter as optimization proceeds.

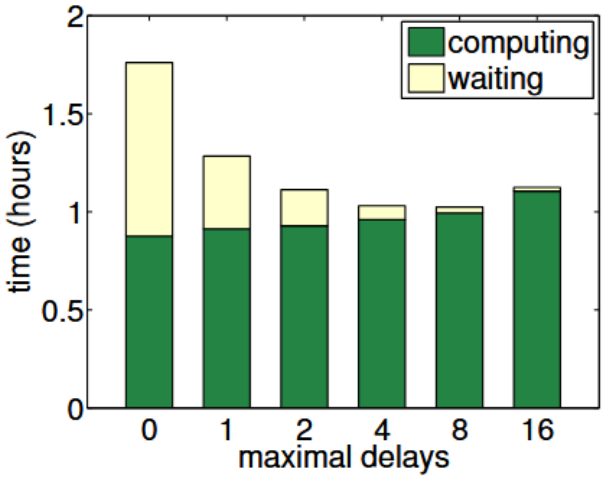


Figure 13: Time a worker spent to achieve the same convergence criteria by different maximal delays.

Latent Dirichlet Allocation

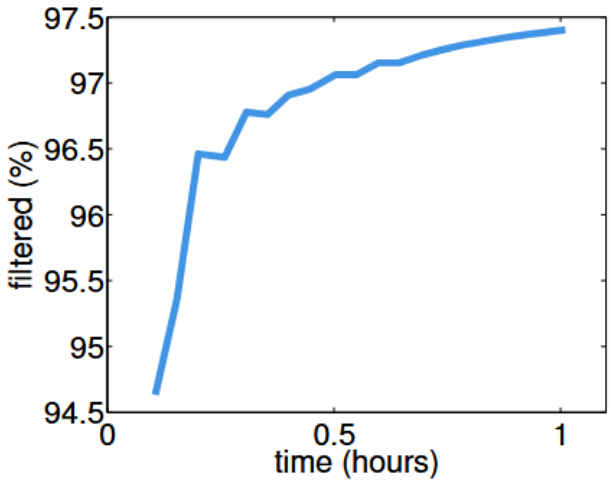


Figure 12: Unique features (keys) filtered by the KKT filter as optimization proceeds.

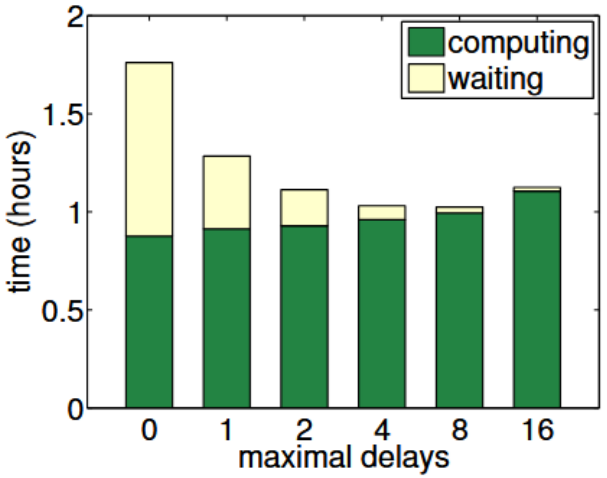


Figure 13: Time a worker spent to achieve the same convergence criteria by different maximal delays.

Sketches

"Very different from Machine Learning" to show generality

Inserting streaming log of pageviews for tracking

Great insert rates!

Recovery of failed nodes within one second

Sketches Network Results

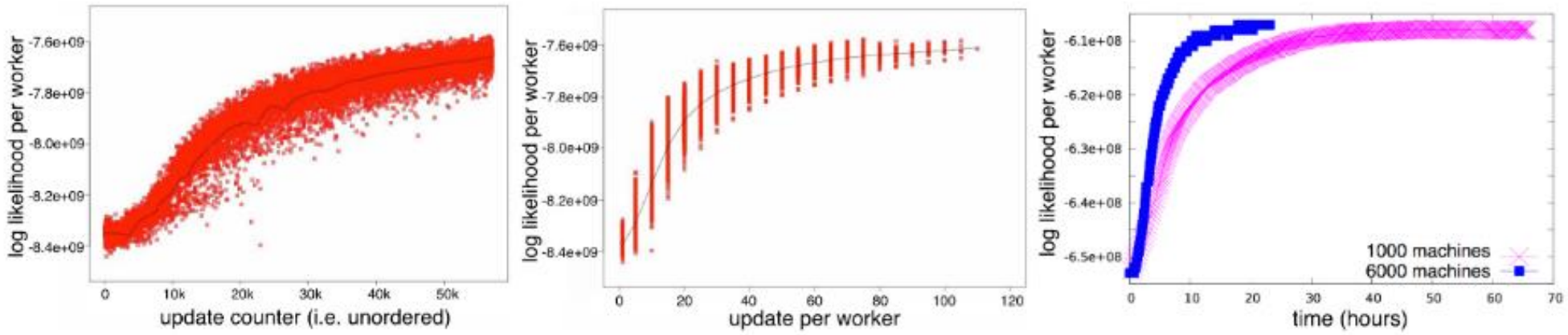


Figure 14: Left: Distribution over worker log-likelihoods as a function of time for 1000 machines and 5 billion users. Some of the low values are due to stragglers synchronizing slowly initially. Middle: the same distribution, stratified by the number of iterations. Right: convergence (time in 1000s) using 1000 and 6000 machines on 500M users.

Citations

1. Li, M., Andersen, D. G., Park, J. W., Smola, A. J., Ahmed, A., Josifovski, V., Long, J., Shekita, E. J., & Su, B.-Y. (2014, October). *Scaling distributed machine learning with the parameter server.* In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)* (pp. 583–598). USENIX Association. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/li_mu
2. https://images.tpointtech.com/ds/images/types-of-sparse-matrices.png
3. Kwok, Y.-K., & Ahmad, I. (1996). *Dynamic critical-path scheduling: An effective technique for allocating task graphs to multiprocessors.* **IEEE Transactions on Parallel and Distributed Systems, 7**(5), 506–521. https://doi.org/10.1109/71.503776
4. https://www.geeksforgeeks.org/consistency-model-in-distributed-system/
5. https://upload.wikimedia.org/wikipedia/en/thumb/8/80/Wikipedia-logo-v2.svg/1200px-Wikipedia-logo-v2.svg.png
6. Li, M., Andersen, D. G., Smola, A., & Yu, K. (2014). Communication efficient distributed machine learning with the parameter server. Advances in Neural Information Processing Systems, 27.